Infrastructure as Code with Terraform on Azure

From on-premise datacenters, to the cloud, to infrastructure you can write, review, and version like software.

What this course covers

01 **Foundations**
Why infrastructure as code

02 **Install & core workflow**
Write, plan, apply

03 **The HCL language**
Blocks, variables, functions

04 **State & workspaces**
Terraform’s record of reality

05 **Modules & reuse**
Package and share infrastructure

06 **Providers & credentials**
Authentication and versioning

07 **Day-2 operations**
Provisioners, drift, import, debugging

08 **Security & cost**
Protect secrets, control spend

09 **Delivery**
Environments, testing, CI/CD

10 **Adoption at scale**
Individual to organisation

Why, what, and how

WHY

The old way hurt

How servers were procured on-premise, and why it was slow, costly, and error-prone.

WHAT

Cloud, then code

How the cloud made infrastructure on-demand — and why we then chose to describe it as code.

HOW

Terraform

One language for every provider, and a write → plan → apply workflow you run yourself.

Procuring a server, the on-premise way

When a business owner raised a new requirement, analysts and product owners turned it into an infrastructure change — **a new virtual machine, more storage, a bigger database server.** That request then travelled, by hand, through a chain of specialist teams before a single application could be deployed.

7 separate hand-offs between teams for one request

weeks

— often months — from request to a usable server

One request, seven hand-offs

01
Business owner
raises a new requirement

02
Analyst / PO
translate to infra needs

03
Procurement
approve & purchase

INFRASTRUCTURE TEAM · THE SLOW MIDDLE

04
Field engineer
reuse or rack a new server

05
Network admin
attach it to the network

06
Storage admin
assign storage to the VM

07
Application team
finally deploys the app

TYPICAL ELAPSED TIME				
	1–2 wks	**2–4 wks**	**days**	**finally**
	Approvals & purchase order	Procure & rack hardware	Network & storage config	Hand-off to app team

Slow, costly, and error-prone by design

SLOW

Weeks of waiting

Approvals, purchase orders, shipping, racking and cabling — the request queues at every stage.

COSTLY

Capital up front

Hardware is bought and paid for years ahead, long before the application earns anything back.

WASTEFUL

Sized for the peak

Capacity is provisioned for the busiest day, then sits mostly idle the rest of the year.

INCONSISTENT

Snowflake servers

Every box is configured by hand, so no two match. Human error becomes drift, drift becomes outages.

RIGID

Hard to change

Scaling down means de-racking hardware; even a small tweak reopens the whole ticket chain.

OPAQUE

No audit trail

What changed, when and by whom lives in people’s heads and email threads — not a record.

The cloud made infrastructure on-demand

BEFORE · ON-PREMISE

Buy and own the hardware

Size for peak, pay up front

Weeks of hand-offs to provision

Capacity is fixed once racked



AFTER · CLOUD

Rent capacity from a provider

Pay only for what you use

Provision in minutes over an API

Scale up or down on demand

Clicking through consoles doesn’t scale

The cloud removed the wait, but manual, point-and-click provisioning brought back the old enemy in a new form: **no record of what was created, why, or by whom.**

Configuration drift — the live environment slowly diverges from what anyone documented.

Not repeatable — standing up an identical staging or DR copy means clicking it all again, from memory.

No review or audit — a change in production leaves no diff, no approval, no history.

Infrastructure as Code, defined

Describe the infrastructure you want in **plain, version-controlled text files** — then let a tool create, change, and destroy real resources to match.

01 **Write** the desired state in code

02 **Commit** it to Git, review, approve

03 **Apply** — the tool makes it real

Why teams codify their infrastructure

Fast & repeatable

Stand up a whole environment in minutes — and an identical copy just as fast.

Consistent

Dev, staging and production come from the same files — no snowflakes, no drift.

Reviewable

Every change is a pull request — discussed and approved before it touches anything.

Auditable

Git history is the record: what changed, when, by whom, and why.

Self-service & scalable

Teams provision from approved modules; scale up or down by changing one number.

Recoverable

Rebuild an entire environment for disaster recovery or testing, on demand, from code.

Terraform: one language, every provider

DECLARATIVE

You describe the *end state* , not the steps. Terraform computes the difference and makes only the changes needed.

PROVIDER-BASED

Plugins for Azure, AWS, GCP, Oracle Cloud, VMware, Kubernetes and hundreds more — same workflow everywhere.

STATEFUL

Terraform records what it manages in a `state` file, so it always knows the gap between code and reality.

```
$ terraform version
Terraform v1.9.0
on linux_amd64

$ terraform providers
provider[registry.terraform.io/hashicorp/aws]
provider[registry.terraform.io/hashicorp/azurerm]
```

Write, plan, apply

01 · WRITE

Describe it

Declare resources in `.tf` files.

→

02 · PLAN

Preview it

See every add, change and destroy before it happens.

→

03 · APPLY

Make it real

Terraform executes the approved plan.

```
$ terraform init      # download the providers
$ terraform plan      Plan: 3 to add, 0 to change, 0 to destroy.
$ terraform apply     Apply complete! Resources: 3 added.
```

A handful of lines that create a server

```
# main.tf — the desired state
provider "azurerm" {
features {}
}
resource "azurerm_linux_virtual_machine" "web" {
resource_group_name = var .rg_name
size = "Standard_D2s_v5"
name = "web-01"
}
```

Declarative: you state what should exist, not the click-by-click steps to build it.

Idempotent: run it twice and nothing changes — the server is created once.

Versioned: this file lives in Git and *is* the source of truth for that server.

Manual hand-offs → on-demand cloud → infrastructure as code

Weeks → minutes

Snowflakes → consistency

Clicks → reviewable code

NEXT · MODULE 02 — HANDS-ON LAB

Install Terraform, write your first `main.tf`, and run `plan` and `apply` against a live provider.

Install & the core workflow

Get Terraform running, then master the write → plan → apply loop.

Install | Write · plan · apply | Provision | Update | Destroy

What it takes to install Terraform

01

Terraform CLI

One static binary — no runtime or agents to run.

02

Provider credentials

API keys for your cloud, via env vars or a config file.

03

A working directory

A folder of .tf files — where you run every command.

04

Network access

init downloads provider plugins from the registry.

```
# Ubuntu — install from the HashiCorp repo
$ sudo apt-get install -y gnupg software-properties-common
$ sudo apt-add-repository \
"deb https://apt.releases.hashicorp.com $(lsb_release -cs) main"
$ sudo apt-get install -y terraform
$ terraform -version Terraform v1.9.0
```

The Terraform workflow

You describe infrastructure as code, then move through one short, repeatable loop. Terraform compares your files against the recorded state and changes only what is needed — the same commands whether you are **creating, updating, or removing** infrastructure.

01
Write
Author resources in .tf files

→

02
Init
Download providers & modules

→

03
Plan
Preview what will change

→

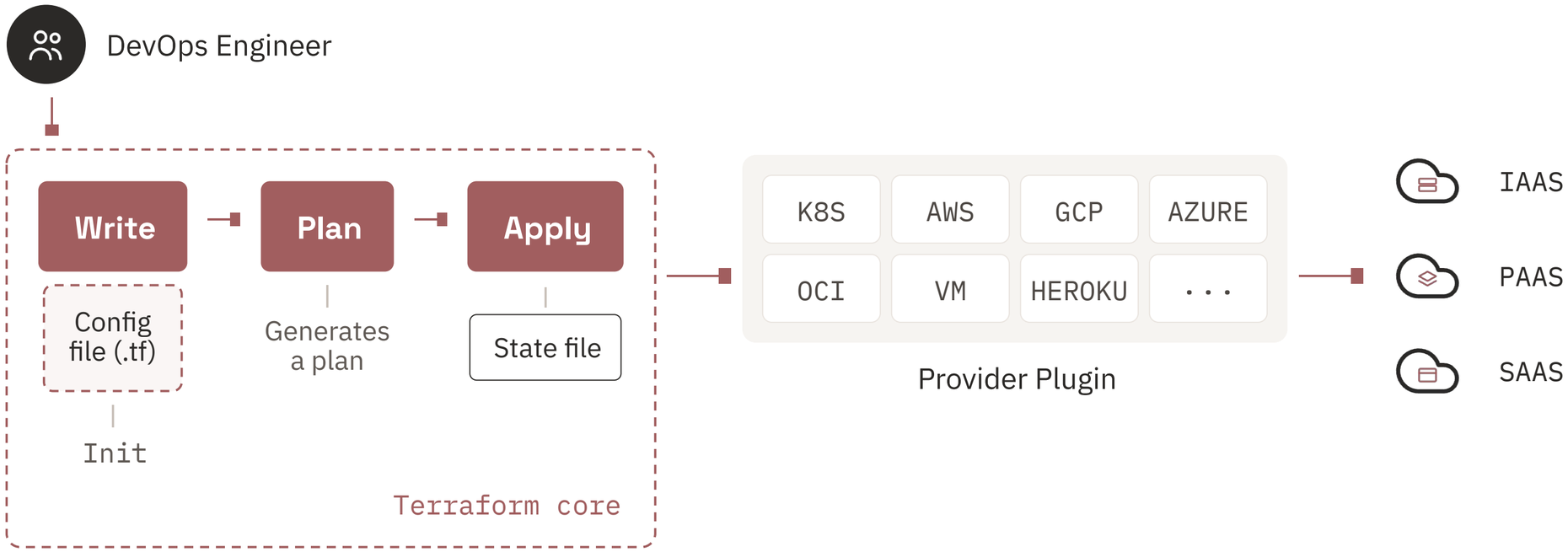
04
Apply
Make the changes real

→

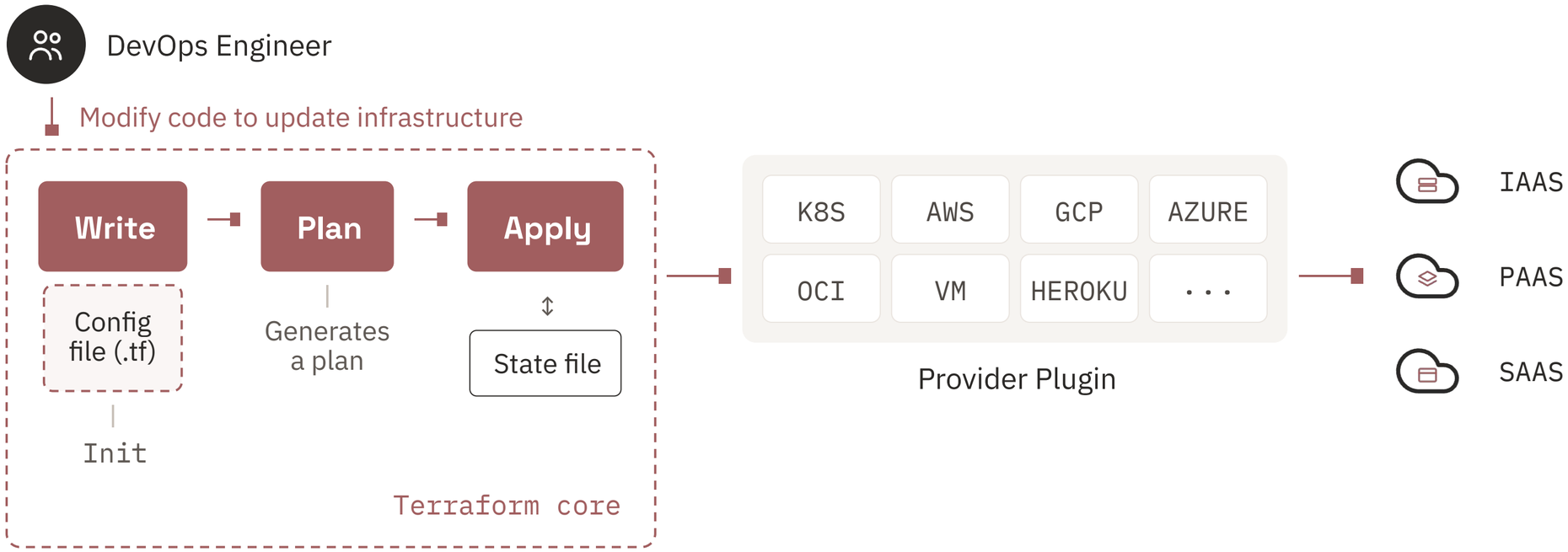
05
Destroy
Tear it all down when done

Every change repeats `plan` → `apply`; run `destroy` when the environment is no longer needed.

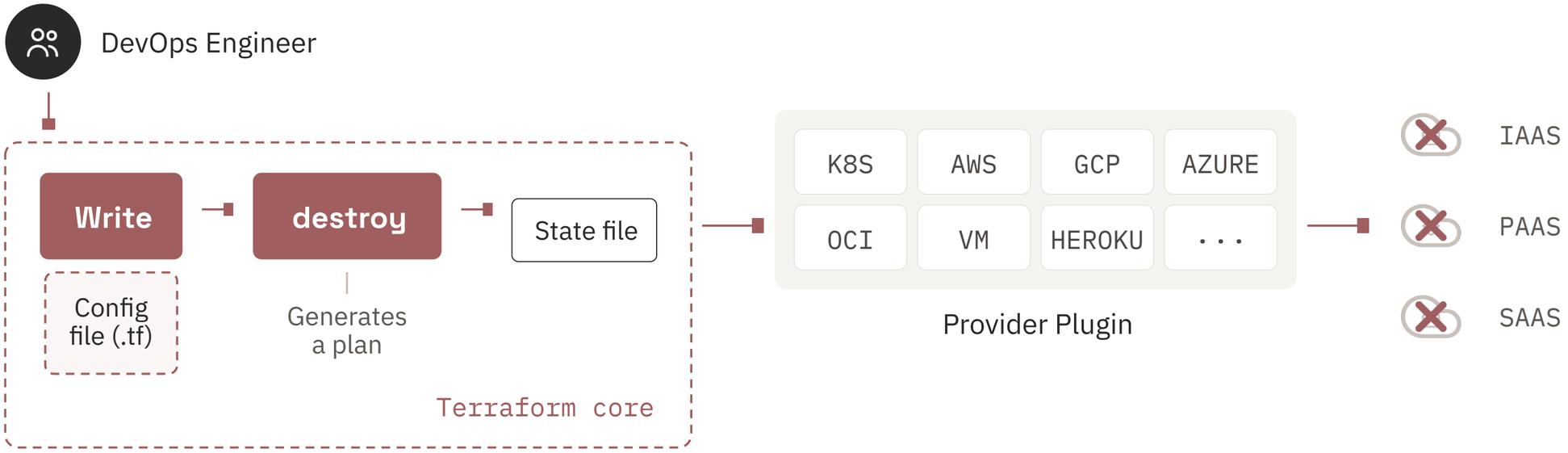
The full workflow, end to end



Changing infrastructure that already exists



Tearing the whole environment down



The Terraform commands we've used

`$ terraform -version`	Check the installed Terraform version.
`$ terraform init`	Initialise the directory; download providers & modules.
`$ terraform providers`	Show the providers this configuration needs.
`$ terraform plan`	Preview the changes before anything happens.
`$ terraform apply`	Create or update the real infrastructure.
`$ terraform destroy`	Remove everything Terraform manages.

The HCL language

Everything in Terraform is written in HashiCorp Configuration Language.

Syntax & blocks | Data types | Variables | Expressions & functions | Meta-arguments | Outputs · locals · data

Everything in HCL is a block

```
resource "azurerm_linux_virtual_machine" "web" {
size = "Standard_D2s_v5"
name = "web-01"
}
```

Block type Labels argument = expression

Blocks

A type, optional labels, and a body wrapped in `{ }`.

Arguments

`name = value` pairs inside a block.

Expressions

Literals, references like `var.region`, and function calls.

Comments

`#` or `//` for a line, `/* */` for a block.

The blocks you'll use every day

terraform

Global settings, backend, and required provider versions.

```
required_version
```

provider

Configure a cloud or platform to talk to.

```
provider "azurerm"
```

resource

Create and manage a real object.

```
resource "…" "web"
```

data

Read existing or external information.

```
data "…" "x"
```

variable

Declare a typed input to the configuration.

```
variable "region"
```

output

Expose a value once apply finishes.

```
output "public_ip"
```

locals

Name reusable expressions once.

```
locals { … }
```

module

Package and reuse a group of resources.

```
module "network"
```

The data types you can use

`string`
Text.
`"eastus"`

`number`
Integer or float.
`3 · 2.5`

`bool`
True or false.
`true`

`list(T)`
Ordered sequence.
`["a", "b"]`

`set(T)`
Unique, unordered values.
`toset(["a"])`

`map(T)`
Key / value pairs.
`{ env = "dev" }`

`object({…})`
Fixed-shape record.
`{ name = string }`

`tuple([…])`
Fixed-length, mixed.
`["web", 3, true]`

> `any` accepts any type, and `null` represents the absence of a value.

Defining and using variables

DEFINE · variables.tf

```
variable "region" {
type = string
default = "eastus"
description = "Azure region"
}
```

USE · main.tf

```
resource "azurerm_linux_virtual_machine" "web" {
location = var.region
size = var.size
}
```

SET VALUES -var › *.tfvars › TF_VAR_… › default — highest wins

How variable values are set

Values can come from several sources. When more than one is present, the **higher entry wins**.

▲ HIGHEST

01	**Command line** per-run override	`terraform apply -var="region=eastus"`
02	**Variable file (flag)** explicit file	`-var-file="prod.tfvars"`
03	**Auto-loaded files** picked up automatically	`terraform.tfvars · *.auto.tfvars`
04	**Environment variable** one per shell session	`export TF_VAR_region="eastus"`
05	**Default in block** fallback if nothing set	`default = "eastus"`

▼ LOWEST — else Terraform prompts

Expressions, references, and functions

```
# reference other objects
location = var.region
subnet = azurerm_subnet.main.id
rg = data.azurerm_resource_group.main.name
# interpolate & call functions
name = "web-${var.env}"
tags = merge(local.common, {})
# conditional
count = var.env == "prod" ? 3 : 1
```

References

Dotted paths to other values: var, local, data, resource attributes, module outputs.

Interpolation

Embed expressions inside strings with `${ … }`.

Functions

Built-ins like `max`, `merge`, `join`, `cidrsubnet`.

Conditionals

`condition ? a : b` chooses between two values.

Built-in functions

Terraform ships 100+ **built-in** functions, grouped by what they work on — you call them in expressions, but you can’t define your own.

String

`upper · lower · format · join · replace`

Numeric

`max · min · abs · ceil · floor`

Collection

`length · keys · merge · lookup · concat`

Encoding

`jsonencode · yamlencode · base64encode`

Filesystem

`file · templatefile · fileexists`

Date & time

`timestamp · timeadd · formatdate`

Type & value

`tostring · tonumber · try · coalesce`

Network

`cidrsubnet · cidrhost · cidrnetmask`

Functions by example

```
# strings
upper("dev") → "DEV"
format("web-%02d", 3) → "web-03"
join(",", ["a","b"]) → "a,b"

# numbers & collections
max(3, 5, 1) → 5
length(["a","b"]) → 2
merge({a=1}, {b=2}) → {a=1, b=2}

# handy
coalesce(var.name, "web")
cidrsubnet("10.0.0.0/16", 8, 2) → "10.0.2.0/24"
```

Compose values

Transform and assemble strings, numbers, lists, and maps inline.

Handle the unknown

`coalesce` and `try` supply sensible fallbacks.

Test interactively

Run `terraform console` to try any expression before using it.

Meta-arguments on any resource

count

Create a fixed number of identical copies; index them with count.index.

```
count = 3
```

for_each

Create one instance per item in a map or set.

```
for_each = var.buckets
```

depends_on

Force an explicit ordering Terraform can’t infer.

```
depends_on = [azurerm_virtual_network.main]
```

lifecycle

Tune create / update / destroy behaviour.

```
prevent_destroy = true
```

Outputs, locals, and data sources

output

Surface a value after apply.

```
output "public_ip" {
value =
azurerm_linux_virtual_machine
.web.public_ip_address
}
```

locals

Name an expression once, reuse it.

```
locals {
name = "web-${var.env}"
}
# use as local.name
```

data

Read something that already exists.

```
data "azurerm_subscription"
"current" {}
# data.azurerm_subscription
# .current.display_name
```

State & workspaces

How Terraform tracks reality — and keeps environments apart.

State & backends

State ⇔ reality

State commands

Workspaces

State and remote backends

Terraform records everything it manages in a **state file** — the map between your code and the real resources. `plan` and `apply` diff against it.

Shared
One source of truth for the team.

Locked
Locking blocks concurrent applies.

Encrypted
Secrets aren’t left in plaintext.

Versioned
History you can roll back to.

REMOTE BACKEND

```
terraform {
backend "azurerm" {
resource_group_name = "tfstate-rg"
storage_account_name = "tfstateprod"
container_name = "tfstate"
key = "prod/main.tfstate"
}
}
# Azure Blob Storage, with native locking
```

Default is a local `terraform.tfstate` — fine solo, risky for teams.

How state ties code to the real world

CONFIGURATION

What you want

```
resource "…instance"
"web" { size = … }
```

apply

plan

STATE FILE

What was recorded

```
…instance.web
→ /subscriptions/…/web
```

apply

refresh

REAL INFRASTRUCTURE

What actually exists



web-01
running

`plan` compares all three; `apply` makes the real world match your config and records the result in state.

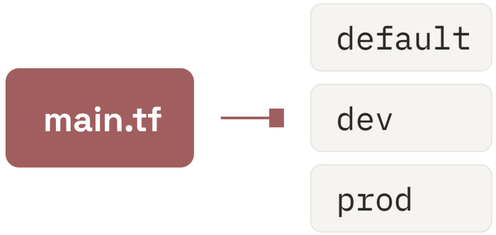
Terraform state commands

`$ terraform state list`	List every resource tracked in state.
`$ terraform state show <addr>`	Show one resource’s recorded attributes.
`$ terraform state mv <a> <b>`	Rename or move a resource within state.
`$ terraform state rm <addr>`	Stop managing a resource — it keeps running.
`$ terraform state pull`	Print the remote state to stdout.
`$ terraform import <addr> <id>`	Bring existing infrastructure under management.
`$ terraform refresh`	Reconcile state with the real world.

Workspaces and environments

One configuration can hold several **named states** — a workspace per environment — so dev, staging, and prod need no duplicated code. The starting workspace is `default`.

`$ terraform workspace list`	List workspaces; * is current.
`$ terraform workspace new dev`	Create and switch to one.
`$ terraform workspace select prod`	Switch to an existing one.




```
display_name =
"web-${terraform.workspace}"
```

> For strong isolation, many teams use separate directories or a `-var-file` per environment instead.

Modules & reuse

Package infrastructure once and reuse it everywhere.

Calling modules

Structure

Inputs & outputs

Modules and reuse

CALL A MODULE

```
module "network" {
source = "./modules/network"
vnet_cidr = "10.0.0.0/16"
}
# reference its output
subnet_id = module.network.subnet_id
```

WHERE MODULES COME FROM

Local path

`./modules/network`

Terraform Registry

`Azure/vnet/azurerm`

Git repository

`git::https://github.com/org/repo.git`

WHY **DRY** **Consistent** **Versioned** **Shareable**

How a module is structured

DIRECTORY LAYOUT

```
project/
├─ main.tf # root module
├─ variables.tf
├─ outputs.tf
└─ modules/
└─ network/ # child module
├─ main.tf # resources
├─ variables.tf # inputs
└─ outputs.tf # outputs
```

`main.tf`

The resources the module creates.

`variables.tf`

Its inputs, declared as variable blocks.

`outputs.tf`

The values it returns to the caller.

Root vs child

The directory you run Terraform in is the root; it calls child modules. Any folder of .tf files is a module.

Passing data in and out

CHILD · modules/network

```
# variables.tf — input
variable "vnet_cidr" {}
# outputs.tf — result
output "subnet_id" {
value = azurerm_subnet.main.id
}
```

ROOT · main.tf

```
module "network" {
source = "./modules/network"
vnet_cidr = "10.0.0.0/16"
}
# read the output
id = module.network.subnet_id
```

> Remote modules take a `version` constraint; add `count` or `for_each` to a module block to create several instances.

Providers & credentials

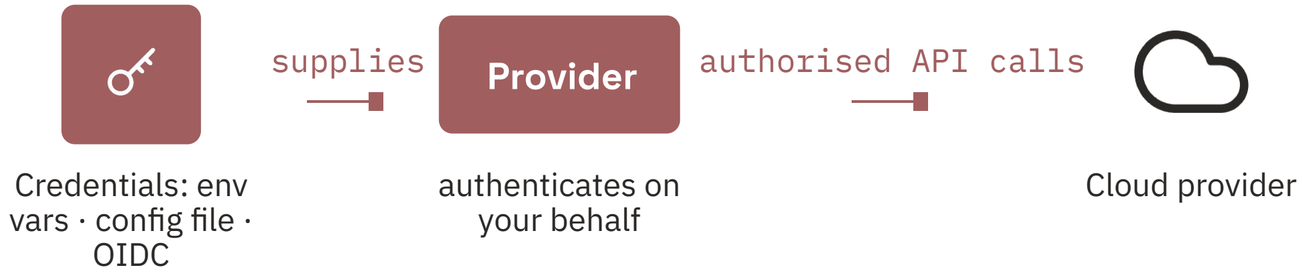
Authenticate to the cloud and pin what you depend on.

Authentication

Version pinning

How the provider authenticates

The provider must **authenticate** before Terraform can touch the cloud — credentials come from a source chain, never hardcoded.



> Prefer **short-lived, dynamic credentials** (OIDC) — no long-lived keys to leak.

Where credentials come from

EXAMPLE · env + provider

```
$ export TF_VAR_region="eastus"
$ export ARM_SUBSCRIPTION_ID="00000000-…"
provider "azurerm" {
features {}
# creds from Azure CLI / OIDC
}
```

Environment variables

Standard cloud vars or `TF_VAR_` inputs — the usual first choice.

Shared config file

An Azure CLI login, AWS credentials file, or CLI profile the provider reads.

Dynamic creds (OIDC)

CI mints short-lived, least-privilege tokens per run — nothing to store.

Pinning provider versions

DECLARE & CONSTRAIN

```
terraform {
required_version = ">= 1.6"
required_providers {
azurerm = {
source = "hashicorp/azurerm"
version = "~> 4.0"
}
}
}
```

CONSTRAINT OPERATORS

`= 5.2.0`	Exactly this version.
`>= 5.0`	This or anything newer.
`~> 5.0`	Allow 5.x, block 6.0 (pessimistic).
`>=5.0, <6.0`	A bounded range.

`.terraform.lock.hcl` records the exact versions and hashes — commit it so everyone resolves the same, and run `init -upgrade` to move within your constraints.

Day-2 operations

Living with infrastructure after the first apply.

Provisioners | Debugging | Drift detection | Import

Provisioners: the last-resort escape hatch

Run extra steps on a resource — a script or a file copy — during create or destroy, for what a provider can’t express declaratively.

`local-exec`	Run a command where Terraform runs.
`remote-exec`	Run commands on the resource, via a connection.
`file`	Copy files onto the resource.

```
resource "…instance" "web" {
provisioner "remote-exec" {
inline = ["sudo dnf install nginx"]
connection {
type = "ssh"
host = self.public_ip
}
}
}
```

Last resort. Prefer `cloud-init` / `user_data`, golden images, or config management — reach for provisioners only when there is no other way.

Debugging and TF_LOG

TURN ON LOGGING

```
$ export TF_LOG=DEBUG
$ export TF_LOG_PATH=./tf.log
$ terraform apply
$ unset TF_LOG # turn it off
```

`TRACE` › `DEBUG` › `INFO` › `WARN` › `ERROR`

most → least verbose

OTHER AIDS

`terraform validate`
Catch configuration errors early.

`terraform console`
Evaluate expressions interactively.

`TF_LOG_CORE / _PROVIDER`
Scope logging to core or providers.

`crash.log`
Written automatically on a panic.

When reality drifts from your code

Drift is when the **real infrastructure no longer matches** your code and state — usually from a manual change made outside Terraform.

TERRAFORM EXPECTS

```
size = "D2s_v5"
vcpus = 2
```

what code & state record

DRIFT

ACTUALLY RUNNING

```
size = "D8s_v5"
vcpus = 8
```

changed by hand in the console

> `terraform plan` re-reads the real world and surfaces the difference as a proposed change.

Detecting and correcting drift

DETECT

`$ terraform plan` Refreshes state, shows drift as changes.

`$ terraform plan -refresh-only` See drift alone, no config changes.

CORRECT

Revert

Re-apply to push reality back to the code.

Accept

Update the code to match, then apply.

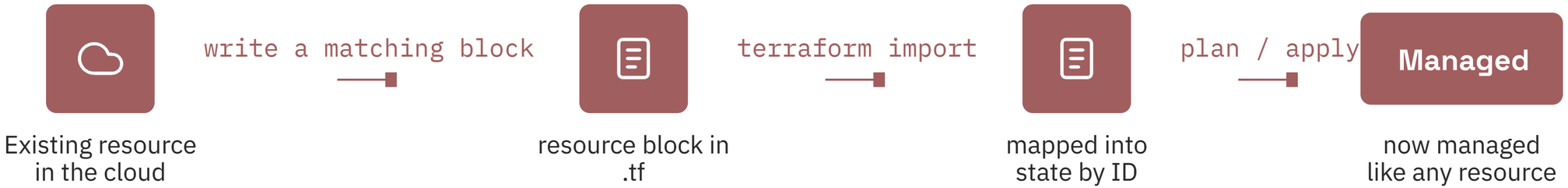
Record

`apply -refresh-only` writes it to state.

Prevent it: make every change through Terraform and lock down console access. HCP Terraform can run **continuous drift detection** and alert admins automatically.

Adopting existing infrastructure

Resources created by hand or another tool can be **brought under Terraform** — without recreating them.



> Import **records** the resource in state — it never creates or changes the real object.

Two ways to import

CLI COMMAND

```
# write the resource block first
$ terraform import \
azurerm_linux_virtual_machine.web \
/subscriptions/…/web
```

IMPORT BLOCK · v1.5+

```
import {
to = azurerm_linux_virtual_machine.web
id = "/subscriptions/…/web"
}# reviewable in Git
```

Gotchas: your block must match reality or the next apply proposes changes; import is one resource at a time; `plan -generate-config-out` can scaffold the config.

Security & cost

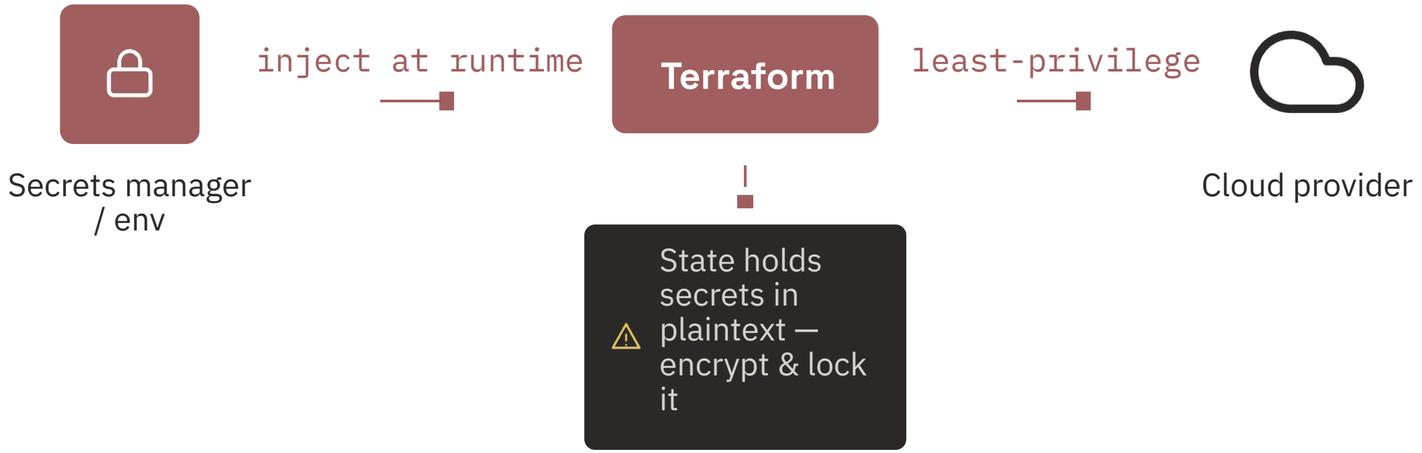
Protect secrets and keep spend under control.

Secrets

Cost management

Handling credentials and secrets

Keep secrets **out of your code and out of reach** — injected at runtime, scoped tightly, and never committed to Git.



> **Never** hardcode secrets in `.tf` files or commit them to Git.

Secure Terraform in practice

Mark it sensitive

`sensitive = true` redacts values from plan, output, and logs.

Inject at runtime

Read from a secrets manager (Vault, Azure Key Vault) via a data source, or from `TF_VAR_` env vars.

Protect the state

State holds secrets in plaintext — encrypt the remote backend and restrict access.

Least-privilege creds

Scope provider roles tightly; prefer short-lived, dynamic credentials over static keys.

Add `*.tfstate`, `*.tfvars`, and `.terraform/` to `.gitignore` so nothing sensitive is ever committed.

See the cost before you apply

Because infrastructure is code, you can **price a change before applying it** — on the pull request, not the invoice.



> Reviewers approve the **cost**, not just the code — a policy can block changes that exceed budget.

Keeping spend under control

Estimate on every PR

A cost tool reads the plan and shows the monthly delta before merge.

Tag everything

Attribute spend by `team`, `env`, and cost centre — enforce tags in policy.

Right-size & tear down

Prefer smaller default VM sizes; destroy idle, ephemeral environments — a one-liner with IaC.

Budget guardrail

Policy as code fails a plan that would exceed a spend threshold.

> Cost becomes a **shift-left check** — caught in review, not discovered at month-end.

Delivery

Ship infrastructure safely and repeatably.

Environments

Testing

CI/CD

Laying out dev, staging & prod

Three common ways to separate environments — trading **isolation** against **duplication**.

WORKSPACES

```
# one config
main.tf
terraform workspace
select prod
```

DIRECTORY PER ENV

```
envs/
dev/ # own backend
stg/
prod/
modules/
```

LAYERS / WRAPPERS

```
modules/ # shared
live/
dev/main.tf
prod/main.tf
# thin wrappers
```

> **Directory-per-environment** is the most common: each env has its own state, backend, and variables — explicit and hard to mix up.

Choosing a layout

Workspaces

One codebase, a state per workspace. DRY and quick — but a wrong `select` can hit prod.

Directory per env

Each environment its own folder, backend, and tfvars. Fully isolated; some duplication.

Same modules everywhere

Promote the identical modules dev → stg → prod; only the inputs differ.

Differences in tfvars

Keep env differences in `*.tfvars`, never in forked copies of the code.

Rule of thumb: isolate production's state and credentials, and change environments by inputs — not by branching the configuration.

A ladder of checks

Layer your checks from **fast & cheap** to **slow & thorough**.

▲ MORE THOROUGH

`terraform fmt / validate`	Formatting and syntax — instant, every save.
`terraform plan (review)`	See exactly what would change before applying.
`policy as code`	Sentinel / OPA check the plan against your rules.
`terraform test`	Native tests spin up resources and assert on them.
`Terratest / end-to-end`	Deploy, verify with real calls, then destroy.

▼ FASTER, RUN ON EVERY COMMIT

Testing in practice

NATIVE TEST · main.tftest.hcl

```
run "defaults" {
command = plan
assert {
condition = azurerm_linux_virtual_machine
.web.size != ""
error_message = "size required"
}
}
```

fmt & validate

Formatting and config validity — run in every pre-commit hook and CI job.

Validation & preconditions

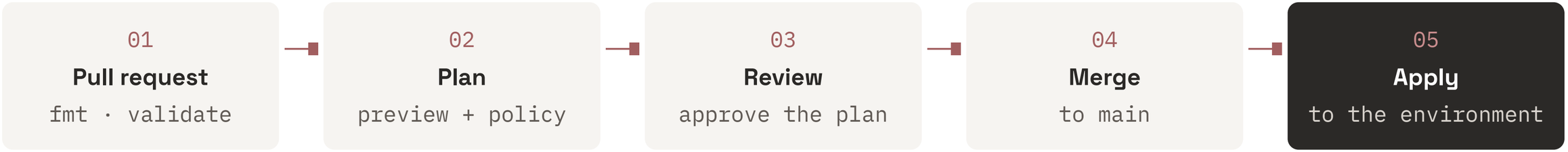
`validation` blocks and `precondition` reject bad inputs before plan.

Wire it into CI

Post plan to the pull request; let policy gate the merge.

Terraform in a pipeline

A pipeline turns pull requests into safe, automated applies — **plan on the PR, apply on merge**.



> Everything before merge is **read-only**; `apply` runs only after review — with policy gating the merge.

What a solid pipeline needs

Plan on PR, apply on merge

Post plan output to the pull request; apply only from the protected main branch.

Remote state + locking

Shared backend with locking so concurrent pipeline runs never collide.

Secrets in CI, short-lived

Credentials from the CI secret store or OIDC — never checked into the repo.

Gate on policy & cost

Fail the check on a policy violation or a budget-busting plan.

> General CI works, or a purpose-built runner — **HCP Terraform, Atlantis, Spacelift** — adds locking, RBAC, and approvals out of the box.

Adoption at scale

From one practitioner to the whole organisation.

Individual | Team | Multi-team | Organisation

Terraform adoption across levels

The **write → plan → apply** loop never changes. What scales up around it is collaboration and governance — from one laptop to the whole organisation.

01

Individual

One practitioner · local state

02

Team

Shared remote state · VCS

03

Multi-team

Module registry · policy as code

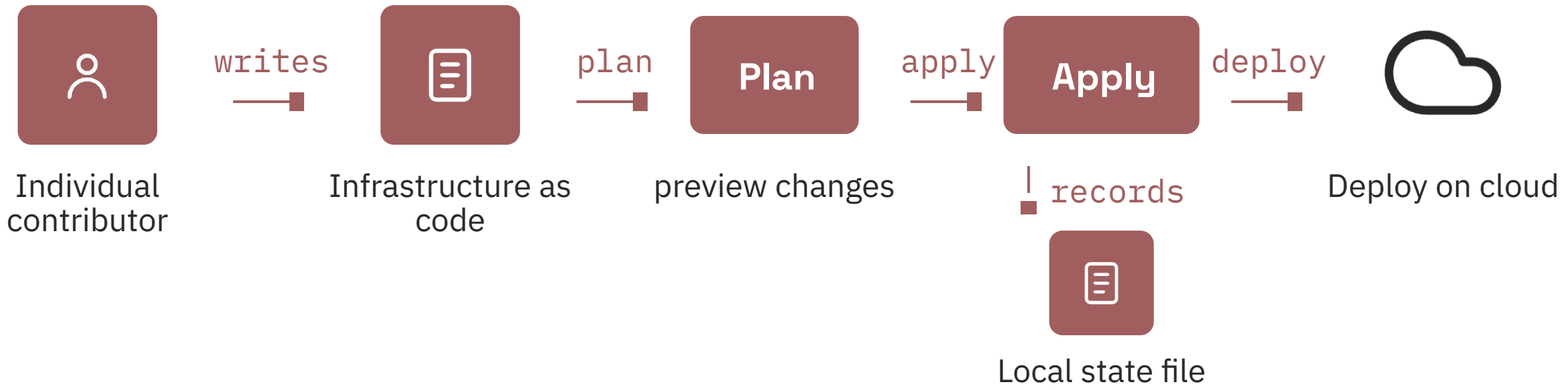
04

Organisation

Platform team · governance at scale

MORE PEOPLE · MORE GOVERNANCE →

Individual practitioner



> One machine, one `terraform.tfstate` — write, plan, apply, then edit and repeat.

The individual workflow, in detail

One person, one configuration, one state file — a fast feedback loop with nothing to coordinate.

Tight local loop

Write config, run plan, run apply, then edit and repeat — seconds, not tickets.

Local state

State sits on your machine. Simple, but yours alone — and easy to lose.

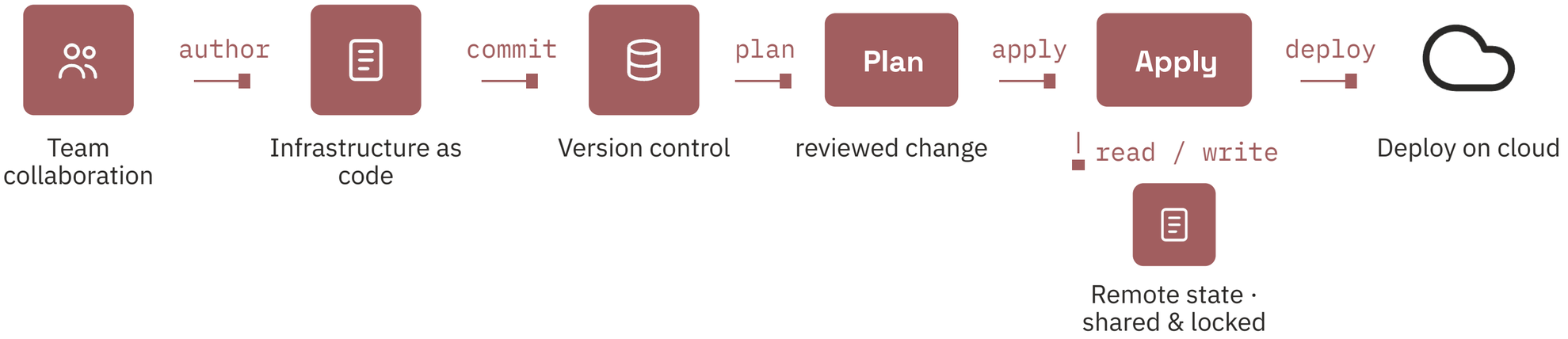
CLI-driven

No remote backend or automation yet — just you and the Terraform binary.

Build good habits early

Pin versions, fmt & validate, and factor patterns into modules so scaling is smooth.

A collaborating team



New pieces vs. the individual: **version control** as the single source of truth, and a **remote, locked state** everyone shares.

The team workflow, in detail

Add a second person and you need a single source of truth and shared, locked state — the loop itself stays the same.

Version control

Config lives in Git as the single source of truth; changes land through pull requests.

Remote state

A shared backend so everyone reads and writes the same recorded reality.

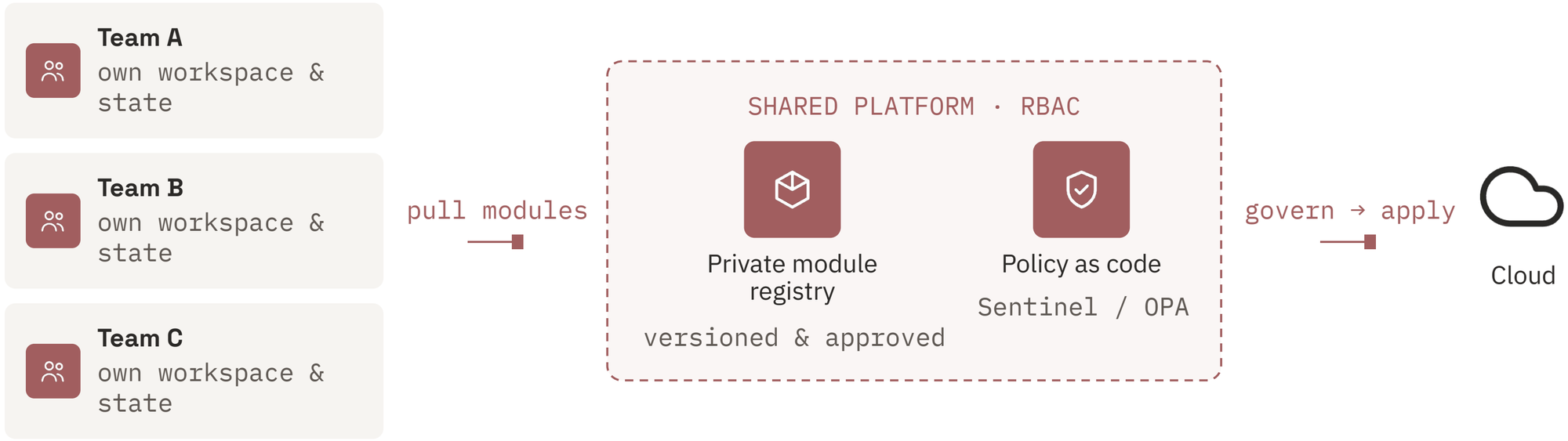
State locking

Prevents two applies at once — no more divergent, conflicting state files.

Same workflow

Write, plan, apply is unchanged — collaboration is simply layered on top.

Multiple teams



Teams share one **approved module registry** and their own isolated state; every plan is **policy-checked** before it reaches the cloud.

Coordinating multiple teams

Many teams need **standards without central bottlenecks** : shared modules, isolated state, and automated policy.

Private module registry

A curated, versioned catalog — teams consume approved modules instead of copy-paste.

RBAC

Separate who can publish from who can consume; scope access per workspace.

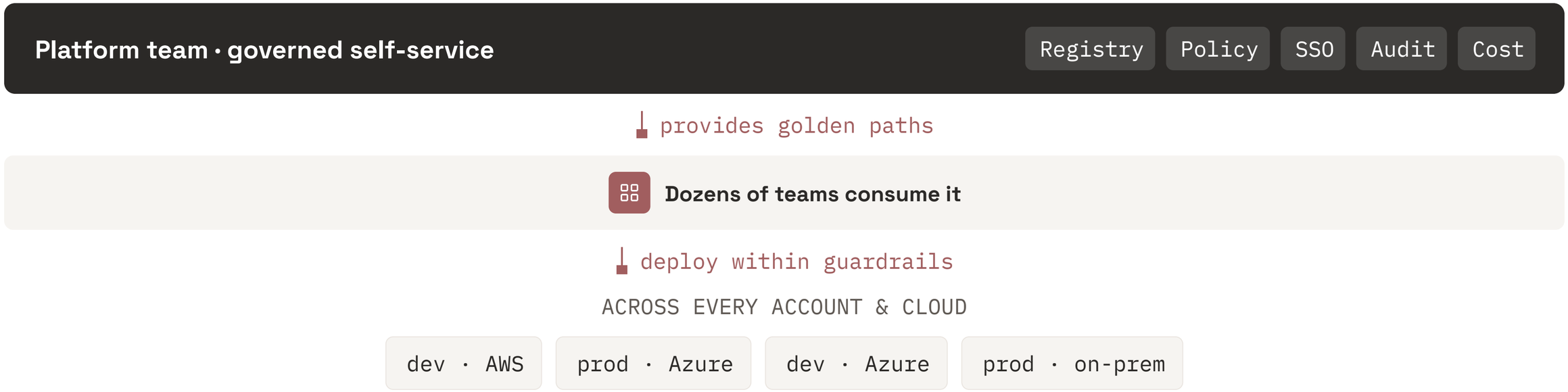
State isolation

A workspace and state per team and environment — a small blast radius.

Policy as code

Sentinel or OPA guardrails check every plan before it is applied.

The whole organisation



Terraform across the organisation

A platform team provides **governed self-service** so every team ships fast within guardrails — across clouds and accounts.

Platform team enables

Golden paths and self-service — enablement, not gatekeeping.

Governance at scale

Org-wide policy, SSO, audit, cost estimation, and drift detection.

Standardisation

One consistent workflow across every team, account, and cloud.

Ownership boundaries

Split state by account and environment — e.g. separate dev and prod.

Thank you

From why infrastructure as code, through the language and workflow, to running Terraform at organisation scale.

Write · plan · apply

Questions welcome